LaTeX 文書作成向けエディタへの手引

[with LuaLaTeX]

ru_museum: GitHub)

更新：2026 年 11 月 6 日

目次

1 概要 1

2 作業環境 1

3 Editors 2

3.1 Kate . . . 2

3.1.1 Snippets . . . 2

3.1.2 External Tools . . . 2

3.1.3 Valiables 一覧 . . . 3

3.2 KDevelop . . . 4

3.2.1 Snippets . . . 4

3.2.2 External Script . . . 4

3.3 Pluma . . . 5

3.3.1 Snippets . . . 5

3.3.2 External Tools . . . 6

3.3.3 Variables 一覧 . . . 7

3.4 VSCodium . . . 8

3.4.1 Snippets . . . 8

3.4.2 Tasks . . . 9

3.4.3 Variables 一覧 . . . 11

3.5 Zed . . . 13

3.5.1 Snippets . . . 13

3.5.2 Tasks . . . 14

4 TIPS 16

4.1 VSCodium：Build の注意点 . . . 16

1 概要

- これは Gedit の新方針による機能低下[*1]に伴い、他エディタへの一**時的**移行を検証する手引です。
  主に LaTeX 文書編集に関連する External Tools(Tasks) 及び Snippets の活用事例を解説しています。
- Gedit 最新版 (v51.0) では Python 依存のプラグインは全て使用不可となっており機能の低下が現実となっています。軽量で使い勝手の良い Gedit には一日も早く元に戻って欲しい所ですが、ここでは **LuaLaTeX 環境で作業を行う上で何れが次善であるか**を各自検証する手掛かりとして下さい。
- 導入を検討する鍵となる機能は、独自のスクリプトを動かせるか否かにあると考えます。
  一連の処理をまとめたスクリプト[*2]を添付し、導入を容易にしています。

2 作業環境

- GNU/Linux Debian 7.2.12-1
- LuaLaTeX / LuaHBTeX 1.24.0 (TeX Live 2026/Debian)
  LaTeX2e <2026-06-01>
- 検証に用いた Editor 一覧 (2026-10)

Name	Version
Kate	26.08.2
KDevelop	6.5.260400 (26.04.0)
Pluma	1.28.0
VSCode	非推奨
VSCodium	1.143
Zed	1.25.3 (2026-10-23)

※ Pluma のプラグイン Snippets の動作不良は最新バージョンで修正されています。
※ VSCode はセキュリティ上の観点[*3]から非推奨ですが、処理手順としては VSCodium と共通していますので参考にして下さい。
※ 最近登場した Zed はこれまで日本語入力が不可でしたが最新版では改善されています。

[*1] ＊内部的に Python への依存を排し C 言語での再構築を行うとしています。
2025-04-30: Some problems currently with Python and gedit plugins
https://gedit-text-editor.org/blog/2025-04-30-no-more-pythons-in-gedit.html
＊Gtk4 への移行に伴い、最近の v48 から組込 plugin の externaltools と snippets を除外することとしています。
基本的に v48 から全ての Python 製のプラグインを除外しています。
詳しくはプラグイン自体の問題ではなく、上流で PyGObject が最新バージョンでサポートされなくなった事が原因らしいです。
2025-04-30 :: No more Pythons in gedit。
https://gedit-text-editor.org/blog/2025-04-30-no-more-pythons-in-gedit.html
Are Third party plugins still going to be a thing ?
https://gitlab.gnome.org/World/gedit/gedit/-/issues/618
[*2] 飽くまで当方の LuaLaTeX 作業環境における処理内容のサンプルですので改変を自由に行えます。
[*3] VSCodium が VSCode から Fork した理由が、telemetry/tracking の問題にあります。

3 Editors

3.1 Kate

- Kate は、KDevelop の元となっていますが使い勝手が若干の異なりますので注意して下さい。

3.1.1 Snippets

(A) Snippets の有効化

- プラグイン Snippets はデフォルトで組込まれています：
  Settings ⇨ Configure Kate ⇨ Plugins ⇨ [Plugin Manager]
  □ Plugins Script & Snippets にチェックを入れます：
  ⇨ 画面右上のツールバーに表示・非表示のアイコンが表示されます。

(B) Snippet の追加・編集

1. 画面右サイドのツールバーのアイコンで表示させます。
2. [Add Repository] で独自の Snippets を追加します。
3. [Add Snippet] 及び [Edit Snippet] で個々に追加・編集を行います。
   ※ 挿入後のカーソル位置は、${cursor}で指示します。
4. [Get New Snippet] で各言語の Snippets をインストール出来ます。

※ Snippet の記述方法は、何れかをインストールして参考にして下さい。

3.1.2 External Tools

(A) Tool の新規作成

1. **Tool の追加と設定：**
   Tool の範型から選択します。
   設定 ⇨ External Tools ⇨ Add ⇨ Add Tool from Defaults ⇨ Tools ⇨ Run Shell Script
   Lualatex による設定例：

項目	入力値
Name	Lualatex Build
Executable	sh
Arguments	-c 'lualatex --shell-escape "%{Document:FileName}" && atril "%{Document:NativePath}/%{Document:FileBaseName}.pdf"'
Input	
Working directory	%{Document:Path}
Mime types	text/x-tex
Save	Current Document
Trigger	After Save
Output	Display in Pane
Editor command	lualatex

- Arguments 欄右端の [ ]（Insert Valiable）をクリックすると Valiable 一覧が表示され入力が出来ます。入力後カーソルを当てると実際値が確認出来ます。
- Mime types はアイコンをクリックすることで選択出来ます。

2. 保存（Ctrl+S 或いは設定した場合ショートカット「Alt+F5」[*4]) でビルドが開始されます。
3. ビルド後に Atril[*5]で PDF が開かれます。

(B) スクリプトの使用

- 添付スクリプト lualatex-build.sh は、ビルドに Biber[*6]の処理を挿入することを目的としています。

1. (A) Tool の**新規作成**を参考に Arguments を以下にします：
   -c './lualatex-build1.sh "%{Document:FileName}"'
   ※ Tool 側で PDF を開く場合は、以下とします：[*7]
   -c './lualatex-build1.sh "%{Document:FileName}" && atril "%{Document:NativePath}/%{Document:FileBaseName}.pdf"'

3.1.3 Valiables 一覧

Valiables	参照内容
Document:FileBaseName	File base name without path and suffix of the current document.
Document:FileExtension	File extension of the current document.
Document:FileName	File name without path of the current document.
Document:FilePath	Full path of the current document including the file name
Document:Text	Contents of the current document.
Document:Path	Full path of the current document excluding the file name.
Document:NativeFilePath	Full document path including file name, with native path separator(backslash on Windows).
Document:NativePath	Full document path excluding file name, with native path separator(backslash on Windows).
Document:Cursor:Line	Line number of the text cursor position in current document (starts with 0).
Document:Cursor:Column	Column number of the text cursor position in current document (starts with 0).
Document:Cursor:XPos	X component in global screen coordinates of the cursor position.
Document:Cursor:YPos	Y component in global screen coordinates of the cursor position.
Document:Selection:Text	Text selection of the current document.
Document:Selection:StartLine	Start line of selected text of the current document.
Document:Selection:StartColumn	Start column of selected text of the current document.

[*4] 設定 ⇨キーボードショートカットを設定 ⇨ アクション名を選択 ⇨カスタム ⇨ Alt+F5 をキーボードで入力
[*5] 推奨のビューワー Atril 以外自由に設定出来ます。
[*6] 文献リスト作成の為のパッケージ。
Biber は変更を反映をさせる為に 2 度のコンパイルを必要とし。生成された.bcf ファイルが存在する場合にスクリプトはコンパイルを再度実行します。
[*7] スクリプト側でコメントアウトする必要があります。PDFViewer Atril は自由に設定出来ます。

Document:Selection:EndLine	End line of selected text of the current document.
Document:Selection:EndColumn	End column of selected text of the current document.
Document:RowCount	Number of rows of the current document.
Document:Variable:<variable>	Expand arbitrary document variables.
Date:Locale	The current date in current locale format.
Date:ISO	The current date (ISO).
Date:<value>	The current date (QDate formatstring).
Time:Locale	The current time in current locale format.
Time:ISO	The current time (ISO).
Time:<value>	The current time (QTime formatstring).
ENV:<value>	Access to environment variables.
JS:<expression>	Evaluate simple JavaScript statements.
PercentEncoded:<text>	Percent encoded text.
UUID	Generate a new UUID.

3.2 KDevelop

- KDevelop は KDE のテキストエディタ「Kate」と内部的にほぼ共通しています。

3.2.1 Snippets

1. 設定の Plugins から Snippets を有効にします。
2. Snippets に追加・登録します。

3.2.2 External Script

- [設定] ⇨ Kate を設定 ⇨ External Tools で追加・削除・編集を行います。
- 設定例 (LuaLatex 使用)

設定項目	**入力値**
Name	Lualatex Build
Command	lualatex --shell-escape %f && atril ”%b.pdf”
Input	Nothing
Output	ignore
Errors	ignore
Save mode	Save Active Document
Show output	√
Output Filter	No Filter
Shortcut	メイン：Alt+F5　　代替：なし

- Command: コンパイル後に PDF を開きます。PDFViewer には Atril をお奨めします [*8]。

[*8] evince にはコンパイル時に CPU を過剰に占有しシステムを停止するという重大なバグがあります。

- 「Shortcut：メイン」は自由に設定出来ます（組合せが必要）。
- 上記例 Save mode: Save Active Document の場合、コンパイル前の「保存」イベントは必要なく、「Alt+F5」(Shortcut) でのコンパイルと同時にファイルは保存されます。
- %f、%b 等の Command に使用するの設定値は以下を参照して下さい。

記述	内　容
%u	URL of the active document.
%f	local filepath to the active document.
%n	name of the active document, including its extension.
%b	name of the active document without its extension.
%d	path to the directory of the active document.
%p	URL to the project of the active document.
%s	shell escaped contents of the selection in the active document.
%i	PID of the currently running KDevelop process.

3.3 Pluma

- Snippets の不具合は最近のアップデートで解消されています (2025-11-24)。

3.3.1 Snippets

- Snippets の設定及び編集方法は、Gedit のページ[*9]を参照して下さい。
  殆んどの組込変数は PREFIX を PLUMA に変更することで利用可能です。
  例：$GEDIT_SELECTED_TEXT ⇨ $PLUMA_SELECTED_TEXT
- 表示結果のカーソル位置：$0(最終位置)、$1、$2 ...
- 改行コード：\\ ⇨ \\\\ で表記します。
- Python の利用：
  $< python code >

  時刻表示例：2025 年 11 月 22 日 (土) 22 時 04 分 24 秒

```
$<1:
import time;
# returnで文字列を返します
return time.strftime("%Y年%m月%d日(%a) %H時%M分%S秒")
>
```

書式	表記	内容
%d	22	0 埋めした 10 進数で表記した月中の日にち
%m	11	0 埋めした 10 進数で表記した月
%y	25	0 埋めした 10 進数で表記した西暦の下 2 桁

[*9] Apps/Gedit/Plugins/Snippets // GNOME
https://wiki.gnome.org/Apps/Gedit/Plugins/Snippets

%Y	2025	0 埋めした 10 進数で表記した西暦 4 桁
%H	22	0 埋めした 10 進数で表記した時（24 時間表記）
%I	10	0 埋めした 10 進数で表記した時（12 時間表記）
%M	04	0 埋めした 10 進数で表記した分
%S	24	0 埋めした 10 進数で表記した秒
%A	土曜日	ロケールの曜日名
%a	土	ロケールの曜日名（短縮形）
%B	11 月	ロケールの月名
%b	11 月	ロケールの月名（短縮形）
%j	326	0 埋めした 10 進数で表記した年中の日にち（正月が'001'）
%U	46	0 埋めした 10 進数で表記した年中の週番号（週の始まりは日曜日）
%W	46	0 埋めした 10 進数で表記した年中の週番号（週の始まりは月曜日）

3.3.2 External Tools

- External Tools の設定及び編集方法は、Gedit のページ[*10]を参照して下さい。殆んどの組込変数は PREFIX を PLUMA に変更することで利用可能です。
  例：$GEDIT_CURRENT_DOCUMENT_NAME ⇨ $PLUMA_CURRENT_DOCUMENT_NAME
- 外部ツールマネージャ (Manage External Tools) におけるスクリプト使用例：

  (1)「ツール」で新規作成「LuaLatex Build」として以下を入力します。

設定項目	入力値
ショートカットキー	<Alt>F5
保存	現在の文書
入力	現在の文書
出力	下端のペインを表示
適用範囲	ローカルファイルのみ

  (2)「編集」欄にスクリプトを入力します。

```
#!/bin/sh

FILENAME="$1" # 引数から実行ファイル取得
LUA_CMD="lualatex --shell-escape"

# 生成されるファイル
FILE_PDF="`basename "$FILENAME" .tex`.pdf"
FILE_BCF="`basename "$FILENAME" .tex`.bcf"

# LUALATEX の実行
if [ -e $1 ]; then

  echo "\n=====" $1 "の処理を開始します："
  $LUA_CMD $FILENAME
  echo "処理は正常に終了しました。PDFファイルが開かれます：\n"
```

[*10] Apps/Gedit/Plugins/ExternalTools // GNOME
https://wiki.gnome.org/Apps/Gedit/Plugins/ExternalTools

```
else

  echo "----------------------------------------"
  echo " ファイル名エラー：[" $1 "]"
  echo "   処理に失敗しました。"
  echo "   ⇒ ファイルが存在するかを確認して下さい。"
  echo "----------------------------------------\n"
  return

fi

# BIBER .BCF が存在する場合：
if [ -e $FILE_BCF ]; then

  echo "BCF File のコンパイルを開始します ------"
  biber $FILE_BCF
  echo "BCF File のコンパイルを終了しました ------"

  echo "処理を反映させる為、再度のコンパイルを行います ------"
  $LUA_CMD $FILENAME
  echo "処理は反映されました ------"

fi


# PDFを開く：IDFViewer（atril）は自由に変更可能です
if [ -e $FILE_PDF ]; then

  atril $FILE_PDF # Atril 以外も利用可能
  echo "$FILE_PDF を開きました"

fi

exit 0
```

(3) ショートカットキー「<Alt>+F5」[*11] で文書は保存され、PDF ファイルが開かれます。

3.3.3 Variables 一覧

Variables	表記
$PLUMA_CURRENT_DOCUMENT_DIR	/your/dirname
$PLUMA_CURRENT_DOCUMENT_NAME	test.tex
$PLUMA_CURRENT_DOCUMENT_PATH	/your/dirname/test.tex
$PLUMA_CURRENT_DOCUMENT_SCHEME	file
$PLUMA_CURRENT_DOCUMENT_TYPE	text/x-tex
$PLUMA_CURRENT_DOCUMENT_URI	file:///your/dirname/test.tex
$PLUMA_CURRENT_LINE	your words
$PLUMA_SELECTED_TEXT	your selected text
$PLUMA_CURRENT_LINE_NUMBER	000
$PLUMA_CURRENT_WORD	your-word
$PLUMA_DOCUMENTS_PATH	/your/dirname/test1.tex /your/dirname/test2.tex
$PLUMA_DOCUMENTS_URI	file:///your/dirname/test1.tex file:///your/dirname/test2.tex

[*11] ショートカットキーは自由に設定可能です。
多くのディストリビューションで同梱されている PDFViewer の Evince は不具合が確認されているため非推奨です。

3.4 VSCodium

- VSCodium は MS VSCode から分岐したオープンソース版で、データ収集などのセキュチティ面を除き全ては機能的に同一ですので、以下に掲載の内容は操作も含め VSCode でも共通です。
- Download: VSCodium(GitHub) https://github.com/VSCodium/vscodium/releases
- ROOT ユーザーでの使用（非推奨）：
  # codium --user-data-dir="/root/.config/Code" --no-sandbox
- 参照：
  Variables reference // VSCode
  Snippets in Visual Studio Code // VSCode

3.4.1 Snippets

- Extension にも幾つか Snippets 用のものはありますが、自作も可能です。
- 作業に伴う最適化したカスタマイズを行います。

(A) latex.json の作成

1. Ctrl+Shift+P でコマンドパレットを開く。
2. snippet と入力。
3. Snippets: Configure Snippets を選択。
4. LaTex(latex) を選択。
5. /.config/Code/User/snippets/latex.json にファイルが生成されます。
   ※ 手動で設置することも出来ます（参照：「(B) 添付 latex.json の使用」）。

(B) 添付 latex.json の使用

- 添付 snippets-template-latex.json を latex.json と変更します。
- /.config/Code/User/snippets/latex.json を置きます。
  或いは必要部分をコピーします（自由に編集可能）。
- 追加された snippet は、語頭に「□」印が付されて候補に列挙されます（例：□ sample）。

作成・編集上の注意点

コード	表記例
$0,$1,$2	挿入時のカーソル位置指定 ($0 は最終位置)
\hspace	\\hspace
改行 \\	改行\\\\\\ (\ = \\\ で表現)

(C) 日時 snippets

- date
  2025 年 11 月 01 日（土）07:34:02

```
// 記述例：
"$CURRENT_YEAR年$CURRENT_MONTH月$CURRENT_DATE日($CURRENT_DAY_NAME_SHORT)
                              $CURRENT_HOUR:$CURRENT_MINUTE:$CURRENT_SECOND"
```

※設定値は以下の valiableslist で確認出来ます。

- valiableslist

表記	Variables	表記内容
2025	CURRENT_YEAR	current year
25	CURRENT_YEAR_SHORT	current year's last two digits
11	CURRENT_MONTH	month as two digits (example '02')
11 月	CURRENT_MONTH_NAME	full name of the month (example 'July')
11 月	CURRENT_MONTH_NAME_SHORT	short name of the month (example 'Jul')
01	CURRENT_DATE	day of the month as two digits (example '08')
土曜日	CURRENT_DAY_NAME	name of day (example 'Monday')
土	CURRENT_DAY_NAME_SHORT	short name of the day (example 'Mon')
08	CURRENT_HOUR	current hour in 24-hour clock format
07	CURRENT_MINUTE	current minute as two digits
22	CURRENT_SECOND	current second as two digits
1761952042	CURRENT_SECONDS_UNIX	number of seconds since the Unix epoch
+09:00	CURRENT_TIMEZONE_OFFSET	current UTC time zone offset as +HH:MM or -HH:MM (example -07:00)

※ VSCodium を日本語表示にしていると日本語での表記となります。

3.4.2 Tasks

(A) tasks.json の基本的使用

- 基本的 Build 例：

```
{
    "label": "Lualatex Build",
    "type": "shell",
    "command": "lualatex",
    "args": [
        "--shell-escape", "--output-directory='${fileDirname}'","${file};",
        "atril", "'${fileDirname}/${fileBasenameNoExtension}.pdf'"
    ],
    "problemMatcher": [],
    "group": {
        "kind": "build",
        "isDefault": true
    }
}
```

1. Ctrl+Shift+B でコマンドパレットを開きビルド名（Lualatex Build）を選択し実行します。
2. lualatex によるビルド後に PDF ファイル（atril）が開かれます。

(B) スクリプトの使用

- スクリプトは、主であるビルド処理に追加処理を挿むことを目的としています。
- このスクリプトは、途中に Biber[*12]の処理を挿入しています。

[*12] 3 ページ *6 を参照。

```
#!/bin/sh

###
# LUALATEX BUILD 処理スクリプト
#  lualatex-build.sh *.tex
##

FILENAME="$1" # 引数から実行ファイル取得
LUA_CMD="lualatex --shell-escape"

# 生成されるファイル
FILE_PDF="`basename "$FILENAME" .tex`.pdf"
FILE_BCF="`basename "$FILENAME" .tex`.bcf"

# LUALATEX の実行
if [ -e $1 ]; then

  echo "\n=====" $1 "の処理を開始します："
  $LUA_CMD $FILENAME
  echo "処理は正常に終了しました。PDFファイルが開かれます：\n"

else

  echo "----------------------------------------"
  echo " ファイル名エラー：[" $1 "]"
  echo "   処理に失敗しました。"
  echo "   ⇒ ファイルが存在するかを確認して下さい。"
  echo "----------------------------------------\n"
  return

fi

# BIBER .BCF が存在する場合：
if [ -e $FILE_BCF ]; then

  echo "BCF File のコンパイルを開始します ------"
  biber $FILE_BCF
  echo "BCF File のコンパイルを終了しました ------"

  echo "処理を反映させる為、再度のコンパイルを行います ------"
  $LUA_CMD $FILENAME
  echo "処理は反映されました ------"

fi

# PDFを開く：TASK 側で開く場合はコメントアウトします
if [ -e $FILE_PDF ]; then

  atril $FILE_PDF # Atril 以外も利用可能
  echo "$FILE_PDF を開きました"

fi

exit 0
```

1. 作業フォルダに lualatex-build.sh を設置します[*13]。
2. $ chmod 744 lualatex-build.sh // 実行権限を与えます[*14]
3. tasks.json への登録：

   （A）スクリプト側で PDF を開く場合：

```
{
    "label": "lualatexbuild", // 名前は自由に付けられます
    "type": "shell",
```

[*13] スクリプトは、Snippet からも作成可能としています。言語モード Latex 環境で□ buildscript で生成されます。作業フォルダごとの設置が容易になります。

[*14] 直接実行する場合：$ ./lualatex-build.sh <filename>

```
    "command": "${fileDirname}/lualatex-build.sh ${file}",
    "problemMatcher": [],
    "group": {
        "kind": "build",
        "isDefault": true
    }
},
```

（B）Task 側で PDF を開く場合[*15] ：

```
{
    "label": "lualatexbuild", // 名前は自由に付けられます
    "type": "shell",
    "command": "${fileDirname}/lualatex-build.sh ${file};",
               "atril", "'${fileDirname}/${fileBasenameNoExtension}.pdf'"
    "problemMatcher": [],
    "group": {
        "kind": "build",
        "isDefault": true
    }
},
```

4. Ctrl+Shift+B でコマンドパレットを開きビルド名を選択するとスクリプトが実行されます。

(C) スクリプトの新規作成

- 添付の Snippet(latex.json) より作成することで、作業フォルダごとの設置が容易になります。

1. /.config/Code/User/snippets/latex.json を置きます。
   或いは、既存の latex.json があれば当該部分を追加します。
2.「新しいテキスト ファイル」を作成し右下にある「言語モードの選択」で「LaTeX」を選択します。
3. ”b” を入力して表示された候補 □ buildscript を選択しスクリプトを読込みます。
4. ファイルを lualatex-build.sh として保存し、アクセス権の「実行の許可」を与えます：
   $ chmod 744 lualatex-build.sh
5. スクリプトの実行方法は「**(B) スクリプトの使用 3. tasks.json への登録**」を参照して下さい。

3.4.3 Variables 一覧

(A) 編集上の VARIABLES

Variables	**表記内容**
TM_SELECTED_TEXT	The currently selected text or the empty string
TM_CURRENT_LINE	The contents of the current line
TM_CURRENT_WORD	The contents of the word under cursor or the empty string
TM_LINE_INDEX	The zero-index based line number
TM_LINE_NUMBER	The one-index based line number
TM_FILENAME	The filename of the current document
TM_FILENAME_BASE	The filename of the current document without its extensions
TM_DIRECTORY	The directory of the current document
TM_FILEPATH	The full file path of the current document

[*15] スクリプトの「PDF を開く」部分をコメントアウトする必要があります。

RELATIVE_FILEPATH	The relative (to the opened workspace or folder) file path of the current document
CLIPBOARD	The contents of your clipboard
WORKSPACE_NAME	The name of the opened workspace or folder
WORKSPACE_FOLDER	The path of the opened workspace or folder
CURSOR_INDEX	The zero-index based cursor number
CURSOR_NUMBER	The one-index based cursor number

Variable（参照値）	Description
${userHome}	Path of the user’s home folder
${workspaceFolder}	Path of the folder opened in VS Code
${workspaceFolderBasename}	Name of the folder opened in VS Code without any slashes (/)
${file}	Currently opened file
${fileWorkspaceFolder}	Currently opened file’s workspace folder
${relativeFile}	Currently opened file relative to workspaceFolder
${relativeFileDirname}	Currently opened file’s dirname relative to workspaceFolder
${fileBasename}	Currently opened file’s basename
${fileBasenameNoExtension}	Currently opened file’s basename with no file extension
${fileExtname}	Currently opened file’s extension
${fileDirname}	Currently opened file’s folder path
${fileDirnameBasename}	Currently opened file’s folder name
${cwd}	Task runner’s current working directory upon the startup of VS Code
${lineNumber}	Currently selected line number in the active file

(B) 日時関連の VARIABLES

Variables	表記	表記内容
CURRENT_YEAR	2025	The current year
CURRENT_YEAR_SHORT	25	The current year’s last two digits
CURRENT_MONTH	02	The month as two digits
CURRENT_MONTH_NAME	October	The full name of the month
CURRENT_MONTH_NAME_SHORT	Oct	The short name of the month
CURRENT_DATE	08	The day of the month as two digits
CURRENT_DAY_NAME	Friday	The name of day
CURRENT_DAY_NAME_SHORT	Fri	The short name of the day
CURRENT_HOUR	11	The current hour in 24-hour clock format
CURRENT_MINUTE	03	The current minute as two digits
CURRENT_SECOND	02	The current second as two digits
CURRENT_SECONDS_UNIX	1761878012	The number of seconds since the Unix epoch
CURRENT_TIMEZONE_OFFSET	+09:00	The current UTC time zone offset as +HH:MM or -HH:MM

(C) その他の VARIABLES

Variables	表記内容
RANDOM	6 random Base-10 digits
RANDOM_HEX	6 random Base-16 digits
UUID	A Version 4 UUID
BLOCK_COMMENT_START	Example output: in PHP /* or in HTML <!–
BLOCK_COMMENT_END	Example output: in PHP */ or in HTML –>
LINE_COMMENT	Example output: in PHP //

3.5 Zed

- 日本語入力が不可でしたが修正されています（2026/10）。
- Wayland 環境では未検証です。
- ファイルのコンパイル

  (A) *.tex をコンパイル

  [Run] => [Spawn Task] => lualatex

  (B) 添付スクリプト (lualatex-build.sh) を使用してコンパイル：

  Bottom Dock：

  $ bash ./lualatex-build.sh *.tex

- ROOT での起動

  ROOT 環境での使用はセキュリティ上推奨されてはいませんが可能です。

  # ZED を ROOT で許可する

  Terminal:

  export ZED_ALLOW_ROOT=true

  或いは、

  .bashrc:

  export ZED_ALLOW_ROOT=true

3.5.1 Snippets

LaTex snippets の作成

1. [Ctrl + Shift + P] ⇨ snippets

   (A) snippets: configure snippets

   ⇨ 表示される一覧から目的の言語を選択すると 各.json ファイルが生成されます：

   ※ 全体に共通する global は、snippets.json となります。

   (B) snippets: open folder

   /.config/zed/snippets ディレクトリが開かれ設定済みの snippets が見られます。

2. 右側サイドバーのアイコンで表示・非表示を行います。
3. 自由に追加・削除・編集が可能です。
   上記の設定を行うと以下のファイルが生成されます：
   /.config/zed/snippets/latex.json(選択言語：LaTex)

```
// itemize 環境の設定例
"Itemize": {
  "prefix": "itz",
  "body": [
      "\\begin{itemize}",
      "  \\item $0",
      "  \\item ",
      "\\end{itemize}"
  ],
  "description": "Itemization"
},
```

※ 詳細の記述法は以下を参照して下さい：
Snippets
https://zed.dev/docs/snippets // Zed

3.5.2 Tasks

- 添付スクリプトを Task として登録：
  tasks.json(Run ⇨ Edit tasks.json)

```
{
  "label": "Lualatex-Build", // 処理名
  "command": "$ZED_DIRNAME/lualatex-build.sh", // スクリプトファイル
  "args": ["$ZED_FILE"],      // 実行ファイル名
  "tags": ["bash-script"],    // 分類
  "save": "current"           // ファイルを保存して実行
},
```

【実行】
Shift + Alt + T : [Run] ⇨ [Spawn Task] の候補として Lualatex-Build が表示されます。
PDFViewer Atril[*16] により PDF が表示されます。
更に KeyMap に登録することで、[Alt + F5] で直接実行が出来ます：
/.config/zed/keymap.json(≡ ⇨ Open Keymap)

```
{
  "context": "Workspace",
```

[*16] 他の PDFViewer も設定可能です。

```
  "bindings": {
      "alt-f5": [ // 割り振るキーはバッティングしない限り自由に設定出来ます
        "task::Spawn",
        { "task_name": "Lualatex Build", "reveal_target": "center" }
      ]
  },
},
```

Task 用変数

Variable（参照値）	内容
$ZED_ROW	現在のカーソル位置の行番号
$ZED_COLUMN	現在のカーソル位置の列番号
$ZED_SYMBOL	カーソル位置にあるシンボル（関数名や構造体名など）
$ZED_SELECTED_TEXT	エディタ上で現在選択されているテキスト
$ZED_WORKTREE_ROOT	現在のワークツリーのルートディレクトリの絶対パス。 (例:/Users/my-user/path/to/project)
$ZED_FILE	現在開いているファイルの絶対パス (例:/Users/my-user/path/to/project/src/main.rs)
$ZED_FILENAME	現在開いているファイルのファイル名 (例:main.rs)
$ZED_DIRNAME	現在開いているファイルのファイル名を除いた絶対パス (例:/Users/my-user/path/to/project/src)
$ZED_RELATIVE_FILE	現在開いているファイルの$ZED_WORKTREE_ROOT への相対パス (例:src/main.rs)
$ZED_RELATIVE_DIR	現在開いているファイルのディレクトリの$ZED_WORKTREE_ROOT への ディレクトリ相対パス。(例:src)
$ZED_STEM	現在開いているファイルの stem (拡張子なしファイル名)(例:main)
$ZED_LANGUAGE	現在開いているバッファの言語 (例:Rust,Python,Shell Script)
$ZED_MAIN_GIT_WORKTREE	メインの git worktree の作業ディレクトリへの絶対パス。
$ZED_CUSTOM_RUST_PACKAGE	(Rust-specific) $ZED_FILE ソースファイルの親パッケージ名。
$ZED_MAIN_GIT_WORKTREE	メインの git worktree の作業ディレクトリへの絶対パス。 通常 $ZED_WORKTREE_ROOT に等しいくなります。 git worktrees にリンクされた場合、元のリポジトリの作業ディレクトリです。
$ZED_CUSTOM_RUST_PACKAGE	(Rust-specific) ソースファイル$ZED_FILE の親パッケージの名前。

4 TIPS

4.1 VSCodium：Build の注意点

- LuaLaTeX では SVG などの画像読込には Inkscape が使われていますが、作業フォルダを正式にメニューの「ファイル（File）」から開かないと Inkscape からファイルへの PATH が正しく通らずエラーとなります。
- PDF ファイルが開かれ Task is running の状態の時には、再度「Alt+F5」での起動は一度 Command Pallet を介して実行されます。
- 添付シェルスクリプトの lualatex-build.sh は、コンパイル時に自由に「追加の処理」を行うことが出来ます。